SOFTWARE DEVELOPMENT PROJECT (SDP)

Session 4 – SRS Documentation (IEEE 830 format)

Software Requirements Specification

ExamEnv: from a teacher's PDF to a verified student container

Project	ExamEnv – Infrastructure-as-Code for Exams
Document version	1.0
Date	29 October 2026
Group No.	Group 1
Team members	Gaurav Shukla – CloudOps and Agents Gaurav Singh – Backend and Research Manya Tiwari – Backend and Research Nandani – ML (Parser)
Course	Software Engineering / SDP – CA-2
Status	Draft for submission

Revision History

Version	Date	Description	Author
0.1	15 Oct 2026	Initial outline from project README	Group 1
0.9	25 Oct 2026	Functional and non-functional requirements added	Group 1
1.0	29 Oct 2026	Final draft for Session 4 submission	Group 1

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, Acronyms and Abbreviations 4

1.4 References 4

1.5 Overview of this Document 4

2. Overall Description 4

2.1 Product Perspective 4

2.2 Product Functions (Summary) 4

2.3 User Classes and Characteristics 5

2.4 Operating Environment 5

2.5 Design and Implementation Constraints 5

2.6 Assumptions and Dependencies 5

2.7 Apportioning of Requirements 5

3. Specific Requirements 6

3.1 External Interface Requirements 6

3.2 Functional Requirements 6

3.3 Non-Functional Requirements 9

3.4 Logical Database Requirements 10

3.5 Future Requirements (Not in Version 1.0) 10

Appendix A: Traceability Matrix 11

Appendix B: Analysis Models 11

Appendix C: Open Issues 11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of ExamEnv, a platform that converts a teacher's exam or lab requirement document (PDF) into a machine-readable YAML specification and automatically provisions identical, verified, containerised environments on student machines. The document follows the structure of IEEE Std 830-1998 and is intended for the project team, the course examiner, and any future maintainers.

1.2 Scope

ExamEnv consists of a teacher web dashboard, a backend hub (FastAPI, PostgreSQL, Celery/Redis), an ML parser that uses a large language model (LLM), and a Python command-line Student Agent that drives Docker. The system will:

- accept a requirement PDF from a teacher and generate a draft YAML environment specification;
- let the teacher review, edit and publish the specification (human-in-the-loop);
- let each student pull the published specification, preview it (dry-run), build and run a Docker container and verify it;
- show the teacher a real-time Ready / Failed / Not Started status for every student, with error logs.

Out of scope for version 1.0: grading or submission of student work, remote proctoring, running exams on non-Docker runtimes, and any mobile application. Items listed in Section 3.5 are planned for later releases.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
IaC	Infrastructure-as-Code: describing an environment in a file that a tool can execute repeatably.
SRS	Software Requirements Specification.
YAML	Human-readable data format used for the environment specification.
LLM	Large Language Model used by the ML Parser to read PDFs.
Dry-run	Preview of what will be installed or changed, shown before anything is executed.
Student Agent	Python CLI that runs on the student's machine and talks to the backend and Docker.
Spec	An environment specification (YAML) for one exam or lab.
JWT	JSON Web Token used for authenticated API access.
REST / API	Web interface exposed by the FastAPI backend.
WebSocket	Two-way connection used to push live status updates to the dashboard.
Celery / Redis	Task queue and message broker for asynchronous PDF parsing.

Term	Meaning
TA	Teaching Assistant.

1.4 References

- ExamEnv project README (architecture, data flow, roadmap).
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- Docker Engine, FastAPI, PostgreSQL, Celery and Redis official documentation.
- Companion deliverables: Session 5 (Use Case Diagram and Narratives) and Session 6 (Class Diagram).

1.5 Overview of this Document

Section 2 gives the overall description of the product. Section 3 lists the specific requirements: external interfaces, functional requirements, non-functional requirements, data requirements and future requirements. The appendices contain a traceability matrix and open issues.

2. Overall Description

2.1 Product Perspective

ExamEnv is a new, self-contained system. It is organised into four cooperating zones, which are the major components referred to throughout this SRS.

Zone	Component	Responsibility
Zone 1	Teacher Dashboard (frontend)	Upload PDFs, review/edit YAML, publish specs, view live student status and error logs.
Zone 2	Backend Hub (FastAPI, PostgreSQL, Celery/Redis)	Authentication, file storage, asynchronous jobs, YAML CRUD, status API, WebSocket broadcasting.
Zone 3	ML Parser	Extract packages, versions, environment variables, ports, datasets and setup commands from the PDF and produce YAML.
Zone 4	Student Agent (CLI) + Docker Engine	Pull spec, dry-run, build and run container, run verification, report result.

The system relies on external components: an LLM service for parsing, Docker Engine on every student machine, and local disk or AWS S3 for file storage.

2.2 Product Functions (Summary)

- User management and role-based login (Teacher, Student, Admin).
- Exam creation and requirement-document upload.
- Asynchronous AI parsing of the PDF into a YAML spec.
- Teacher review, edit, versioning and publishing of the spec.
- Student environment provisioning with dry-run, build, run and automated verification.
- Real-time class monitoring and error-log drill-down.

2.3 User Classes and Characteristics

User class	Description	Technical expertise
Teacher / Lead Instructor	Creates exams, uploads requirement PDFs, reviews and publishes specs, monitors readiness.	Domain expert; comfortable with a web UI; may not know Docker or YAML.
Student	Runs the Student Agent on their own computer to get a ready environment.	Basic command-line use; Docker installed.
Administrator	Manages user accounts and roles, and supervises the platform.	Technical; knows deployment basics.
ML Parser (system actor)	Automated LLM-based service invoked by the backend.	Not applicable.
Docker Engine (system actor)	Runtime that builds images and runs containers on student machines.	Not applicable.

2.4 Operating Environment

- Backend: Linux server or cloud VM running Python 3.10+, PostgreSQL, Redis, Docker Compose.
- Teacher dashboard: latest versions of Chrome, Firefox or Edge.
- Student Agent: Windows 10/11, macOS or Linux with Python 3.10+, Docker Engine running, and network access to the backend.

2.5 Design and Implementation Constraints

- Backend must be implemented with FastAPI and PostgreSQL; asynchronous work through Celery and Redis.
- Environments are Docker containers only; no virtual machines.
- The AI output must never reach students without teacher approval.
- All client-server traffic must use HTTPS in production.
- The project is built by a four-member student team (Group 1) within the semester timeline.

2.6 Assumptions and Dependencies

- Students have permission to install and run Docker on their machine.
- The LLM service is reachable from the backend; if not, the teacher can write the YAML manually.
- Requirement PDFs contain selectable text (scanned image-only PDFs are not guaranteed to parse).
- Exam rooms have network access to the backend, except in the future offline mode.

2.7 Apportioning of Requirements

Requirements marked priority M (Must) are in scope for version 1.0. S (Should) are targeted for 1.0 if time permits. C (Could) are deferred. Features in Section 3.5 are planned for later releases.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Teacher dashboard: login page, exam list, upload form, YAML editor with validation messages, publish button, live student status grid (green = Ready, red = Failed, grey = Not Started) and error-log viewer.
- Student Agent CLI: commands examenv-agent pull, dry-run, provision and report with clear colour-coded text messages and a Yes/No confirmation prompt.

3.1.2 Hardware Interfaces

No special hardware. Student machines need enough CPU, memory and disk to run Docker containers (recommended: 4 GB RAM and 10 GB free disk).

3.1.3 Software Interfaces

Interface	Description
REST API (FastAPI)	JSON over HTTPS between dashboard / agent and backend: auth, upload, spec CRUD, status reporting.
PostgreSQL	Stores users, exams, documents, specs, status records and logs.
Celery + Redis	Queues parse jobs and returns results to the backend.
LLM / ML Parser	Receives extracted PDF text and returns structured data used to generate YAML.
Docker Engine API	Used by the Student Agent to build images, start containers and read logs.
Storage (S3 or local disk)	Stores uploaded requirement PDFs.

3.1.4 Communication Interfaces

HTTPS for REST calls, WSS (secure WebSocket) for live dashboard updates, and Redis protocol between backend and Celery workers inside the private network.

3.2 Functional Requirements

3.2.1 Authentication and User Management

ID	Requirement (the system shall ...)	Priority	Use case
FR-01	Allow a user to register an account with name, email and password.	M	UC01
FR-02	Authenticate users with email and password and issue a time-limited access token (JWT).	M	UC01
FR-03	Enforce role-based permissions for Teacher, Student and Admin on every API call.	M	UC01, UC13
FR-04	Allow users to log out and expire sessions automatically after inactivity.	S	UC01

ID	Requirement (the system shall ...)	Priority	Use case
FR-05	Allow an Admin to create, deactivate and assign roles to user accounts.	M	UC13

3.2.2 Exam and Requirement Upload

ID	Requirement (the system shall ...)	Priority	Use case
FR-06	Allow a Teacher to create an exam with title, course code and start and end time.	M	UC02
FR-07	Allow a Teacher to upload a requirement document; only PDF files up to 20 MB shall be accepted.	M	UC02
FR-08	Store the uploaded file in S3 or local disk and save its metadata in the database.	M	UC02
FR-09	Trigger an asynchronous parse job after upload without blocking the Teacher.	M	UC02, UC03
FR-10	Show the parse job status (Queued, Running, Completed, Failed) to the Teacher.	S	UC02

3.2.3 AI Parsing and YAML Generation

ID	Requirement (the system shall ...)	Priority	Use case
FR-11	Extract the text of the uploaded PDF.	M	UC03
FR-12	Extract packages with versions, environment variables, ports, datasets and setup commands from the text.	M	UC03
FR-13	Generate a YAML specification that follows the ExamEnv schema.	M	UC03
FR-14	Validate the YAML against the schema and save it as a Draft spec linked to the exam.	M	UC03
FR-15	If parsing fails, record the error, notify the Teacher, and allow retry or manual YAML entry.	M	UC03, UC04

3.2.4 Teacher Review and Publishing

ID	Requirement (the system shall ...)	Priority	Use case
FR-16	Display the generated YAML in an editor with syntax and schema validation messages.	M	UC04
FR-17	Allow the Teacher to edit and save the Draft spec.	M	UC04
FR-18	Allow the Teacher to publish a spec; only Published specs shall be visible to Students.	M	UC05
FR-19	Keep a version number and history for every spec; editing a Published spec creates a new version.	S	UC04, UC05
FR-20	Allow the Teacher to start from a verified template instead of an AI draft (future).	C	UC14

3.2.5 Student Environment Provisioning

ID	Requirement (the system shall ...)	Priority	Use case
FR-21	Allow a Student to authenticate from the CLI using their account.	M	UC01
FR-22	List the Published exams available to the Student.	M	UC08
FR-23	Download the Published YAML for the selected exam.	M	UC08
FR-24	Perform a dry-run that lists packages, ports, mounts and setup commands, and require an explicit Yes before continuing.	M	UC09
FR-25	Build a Docker image from the spec, or use a pre-built base image and install packages on top.	M	UC10
FR-26	Run the container with the specified exposed ports and volume mounts.	M	UC10
FR-27	Detect port conflicts before running and report them clearly to the Student.	S	UC10
FR-28	Allow the Student to retry or clean up a failed or old environment.	S	UC10

3.2.6 Verification and Reporting

ID	Requirement (the system shall ...)	Priority	Use case
FR-29	Run a verification script inside the container (for example python --version, pip list, import checks).	M	UC11
FR-30	Compare results with the spec and mark every check as passed or failed.	M	UC11
FR-31	Collect verification logs and error messages.	M	UC11
FR-32	Report the final status (Ready or Failed) and logs to the backend through the API.	M	UC12

3.2.7 Teacher Monitoring

ID	Requirement (the system shall ...)	Priority	Use case
FR-33	Show every enrolled Student's status as Ready, Failed or Not Started on the dashboard.	M	UC06
FR-34	Push status changes to the dashboard in real time using WebSockets.	M	UC06
FR-35	Allow the Teacher to open a failed Student's exact error log.	M	UC07
FR-36	Display summary counts (Ready / Failed / Not Started) for the class.	S	UC06
FR-37	Allow the Teacher to search and filter students by status.	C	UC06

3.3 Non-Functional Requirements

Category	ID	Requirement
Performance	NFR-01	95% of standard API requests (excluding parsing) shall respond within 2 seconds.
	NFR-02	A dashboard status change shall appear within 5 seconds of the agent's report.
	NFR-03	A 20-page PDF shall be parsed to a draft YAML within 2 minutes.
	NFR-04	The system shall support at least 200 students provisioning at the same time.
Security	NFR-05	Passwords shall be stored only as salted hashes (for example bcrypt).
	NFR-06	All network traffic shall use HTTPS/WSS in production; secrets shall be kept in environment variables, not in code.
	NFR-07	Containers shall run without privileged mode and with limited host access.
	NFR-08	Setup commands from a spec shall be shown in the dry-run before execution, because the AI may misread or a PDF may contain unsafe text.
	NFR-09	Uploaded files shall be validated for type and size; students shall only access Published specs.
Reliability	NFR-10	The backend shall be available at least 99% during scheduled exam windows.
	NFR-11	Failed parse jobs shall be retried automatically at least once; agent reports shall be safe to repeat (idempotent).
	NFR-12	The database shall be backed up daily.
Usability	NFR-13	A student shall be able to provision an environment with at most two commands after login.
	NFR-14	CLI and dashboard error messages shall be understandable by non-experts and suggest a next step.
	NFR-15	The dashboard shall use clear colour and text labels so that status is not shown by colour alone.
Portability	NFR-16	The Student Agent shall run on Windows, macOS and Linux with Python 3.10+ and Docker.
	NFR-17	The dashboard shall work on current Chrome, Firefox and Edge.
Maintainability	NFR-18	Code shall be modular (api, models, tasks, websockets, agent, parser) with automated tests covering at least 70% of backend logic.
	NFR-19	The system shall be deployable with docker-compose and checked by a CI pipeline on every push.
Scalability	NFR-20	The API shall be stateless so that more backend instances and Celery workers can be added.

Category	ID	Requirement
Accuracy and Auditability	NFR-21	Teacher review shall be mandatory; target extraction accuracy of at least 85% of fields on sample PDFs.
	NFR-22	Uploads, publishes and status reports shall be logged with user and timestamp.

3.4 Logical Database Requirements

Entity	Key data
User	userId, fullName, email, passwordHash, role, isActive
Exam	examId, title, courseCode, startTime, endTime, teacherId
RequirementDocument	docId, examId, fileName, storagePath, uploadedAt, parseStatus
ParseJob	jobId, docId, state, startedAt, finishedAt, errorMessage
EnvironmentSpec	specId, examId, version, yamlContent, state (Draft/Published), createdAt
EnvironmentInstance / StatusRecord	instanceId, studentId, specId, state (Not Started / Ready / Failed), message, updatedAt
VerificationLog	logId, instanceId, checkName, output, passed, timestamp

The full object model is given in the Session 6 class diagram.

3.5 Future Requirements (Not in Version 1.0)

- Template Library of verified YAML templates for common course types.
- Multi-format ingestion: plain text and slide documents in addition to PDF.
- Finer role-based access: Teaching Assistant, Lead Instructor, Admin.
- Environment expiry and automatic cleanup after the exam window.
- Analytics dashboard of common failure reasons across terms.
- Offline / air-gapped mode using pre-downloaded base images.
- Email or Slack notification when many students are in Failed status.

Appendix A: Traceability Matrix

Each use case (Session 5) is traced to the functional requirements it satisfies.

Use case	Name	Functional requirements
UC01	Register / Login	FR-01, FR-02, FR-03, FR-04, FR-21
UC02	Upload Requirement PDF	FR-06, FR-07, FR-08, FR-09, FR-10
UC03	Parse PDF to YAML Spec	FR-09, FR-11, FR-12, FR-13, FR-14, FR-15
UC04	Review and Edit YAML Spec	FR-15, FR-16, FR-17, FR-19
UC05	Publish Environment Spec	FR-18, FR-19
UC06	Monitor Class Readiness Status	FR-33, FR-34, FR-36, FR-37
UC07	View Student Error Logs	FR-35
UC08	View and Pull Exam Spec	FR-22, FR-23
UC09	Dry-Run Preview and Confirm	FR-24
UC10	Provision Environment	FR-25, FR-26, FR-27, FR-28
UC11	Verify Environment	FR-29, FR-30, FR-31
UC12	Report Status to Backend	FR-32
UC13	Manage Users and Roles	FR-03, FR-05
UC14	Use Template Library (future)	FR-20

Appendix B: Analysis Models

- Use case diagram and narratives: Session 5 deliverable (docs/session-05-use-case).
- Class diagram: Session 6 deliverable (docs/session-06-class-diagram).

Appendix C: Open Issues

- Choice of LLM provider and prompt format (TBD by Nandani).
- Final YAML schema fields and versioning rules (TBD by Nandani, Gaurav Singh and Manya Tiwari).
- Storage choice for the first release: local disk or AWS S3 (TBD by Gaurav Singh, Manya Tiwari and Gaurav Shukla).